

MT-PR2 SS2024	Laborprüfung Programmieren 2	Prof. Dr. Zhen Ru Dai
Name / MatrikelNr.		

Aufgabe	1	2	3	4	Summe	Note
Maximum	50	50	40	40	180	
Erreicht						

Organisatorische Hinweise:

- Dauer der Laborprüfung: 3 h.
- Füllen Sie den Zettel an Ihrem Platz aus und unterschreiben Sie diesen.
- Loggen Sie sich auf Ihr Prüfungsaccount ein und verwenden Sie CLion für die Bearbeitung der Aufgaben.
- Folgende Materialien stehen Ihnen als pdf-Dateien zur Verfügung: Prüfungsaufgaben, C++ Buch von Breymann, PR2 Vorlesungsskript, UML Referenzliste.
- Sie dürfen ein DINA-4 Notizblatt beidseitig handschriftlich geschrieben mitbringen.
- Es dürfen außer dem Prüfungsrechner keine elektronischen Geräte benutzt werden.
- Legen Sie Ihren Studierendenausweis gut sichtbar auf den Tisch.
- Melden Sie sich bitte per Handzeichen, wenn Sie zur Toilette müssen (zeitgleich darf nur ein Klausurteilnehmer den Raum verlassen).
- Vermeiden Sie möglichst alle Störungen!

Hinweise zur Prüfung:

- Tragen Sie in der **main.cpp** Ihren Namen und MatrikelNummer ein.
- Lesen Sie sich die Aufgaben ganz genau durch und studieren Sie die Diagramme!
- Achten Sie auf die Kommentarinhalte in den Code-Skeletten.
- Sie dürfen ausschließlich die angegebenen Bibliotheken nutzen.
- Ihre C++ Implementierungen müssen mit den UML-Spezifikationen aus der Aufgabenstellung übereinstimmen.
- Konstruktoren, Destruktoren, Setter- und Getter-Methoden sind nicht in UML modelliert. Sie sind von Ihnen sinnvoll zu definieren.
- Achten Sie bei Ihren Implementierungen darauf, dass Sie möglichst auf Objekt-Referenzen arbeiten. **Diese sind in den Diagrammen nicht modelliert!**
- Sie dürfen bei Bedarf eigene private Methoden implementieren.
- Verändern Sie nicht die Struktur des Verzeichnisbaums!
- Achten Sie auf Ihren Coding Style.
- Folgende Standard-Bibliotheken dürfen benutzt werden: <iostream>, <fstream>, <list>, <string>, <cmath>, <cstdlib>, <cstdio>, <climits>, <ctime>, <float>.

Viel Erfolg!

Beschreibung: Raumüberwachungssystem

Ein Raumüberwachungssystem soll die Luftqualität eines Raumes überwachen. Dazu gehören die Überwachung der Raumfeuchtigkeit und des CO2 Gehalts.

Für die Implementierung des Raumüberwachungssystems werden folgende Klassen benötigt:

- 1) Die optimale Raumfeuchtigkeit in Wohnräumen liegt bei 40-60%. Die Klasse **Humidifier** sorgt dafür, dass der Motor des Luftbefeuchters abhängig von Raumfeuchtigkeit an- bzw. ausgeschaltet wird.
- 2) Die Klasse **HumidFSM** definiert die Steuerungslogik von *Humidifier*.
- 3) Um die Qualität des Systems zu sichern, werden automatisierte Tests benötigt, die vorgefertigte Testdaten aus einer Datei auslesen können. Die Klasse **FileHandler** sorgt für einen reibungslosen Datenimport.
- 4) Über eine wohldefinierte Schnittstelle **IDispatcher** sollen sowohl Messung von Raumfeuchtigkeit wie auch der CO2-Konzentration genutzt werden.

Aufgabe 1: OO-Grundlage [50 Punkte]

Die Klasse *Humidifier* sorgt dafür, dass der Motor des Luftbefeuchters an- und ausgeschaltet wird.

- Beim **Befeuchten** des Raums wird das Fenster geschlossen und der Motor des Luftbefeuchters eingeschaltet.
- Beim **Entfeuchten** des Raums wird das Fenster geöffnet und der Motor des Luftbefeuchters ausgeschaltet.
- Im **Normalbetrieb** ist das Fenster geschlossen und der Motor ausgeschaltet.

- a) Entwickeln und implementieren Sie eine Klasse namens *Humidifier* anhand des gegebenen UML-Klassendiagramms.

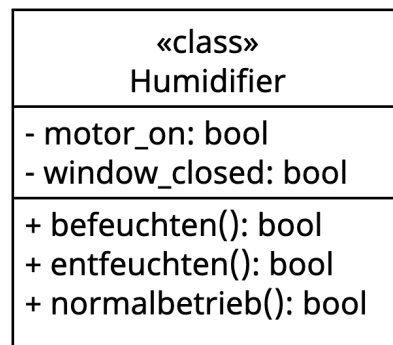


Abbildung 1: Klasse Humidifier

Bei einem neuen Objekt der Klasse *Humidifier* soll der Motor standardmäßig ausgeschaltet und das Fenster geschlossen sein.

bool normalbetrieb():

Beim Normalbetrieb soll das Fenster geschlossen und der Motor ausgeschaltet werden. Zusätzlich soll auf der Konsole folgende Nachricht angezeigt werden: „Raum ok“.

Als Rückgabewert wird der Wert von *motor_on* zurückgegeben.

bool befeuchten():

Beim Befeuchten des Raumes soll das Fenster geschlossen und der Motor eingeschaltet werden. Zusätzlich soll auf der Konsole folgende Nachricht angezeigt werden: „Raum wird befeuchtet“.

Als Rückgabewert wird der Wert von *motor_on* zurückgegeben.

bool entfeuchten():

Beim Entfeuchten des Raumes soll das Fenster geöffnet und der Motor ausgeschaltet werden. Zusätzlich soll auf der Konsole folgende Nachricht angezeigt werden: „Raum wird entfeuchtet“.

Als Rückgabewert wird der Wert von *motor_on* zurückgegeben.

- b) In der Testdatei ***testRaumueberwachungssystem.cpp*** finden Sie zwei vorgefertigte Tests. Diese dienen zur Evaluierung Ihrer Klassen-Implementierung und sollen erfolgreich durchlaufen.

- c) Schreiben Sie einen dritten Test in ***testRaumueberwachungssystem.cpp***, der die ***entfeuchten()***-Methode sinnvoll prüft.

Aufgabe 2: Logik [50 Punkte]

Die Klasse **HumidFSM** definiert die Steuerungslogik von *Humidifier* anhand der angegebenen Zustandsmaschine:

- Nach der Inbetriebnahme des Raumüberwachungssystem wird die Raumfeuchtigkeit überwacht.
- Unterschreitet die Feuchtigkeit den unteren Grenzwert *MIN*, so wird der Raum befeuchtet.
- Übersteigt die Feuchtigkeit den oberen Grenzwert *MAX*, so wird der Raum entfeuchtet.
- Erreicht die Feuchtigkeit wieder Werte zwischen *MIN* und *MAX*, so geht das System wieder in den Normalbetrieb.
- Nach jedem Zustandswechsel wird der aktuelle Zustand auf der Konsole ausgegeben.
- Es gibt keinen direkten Übergang zwischen Befeuchten und Entfeuchten, d.h. nach Be- bzw. Entfeuchten erreicht die FSM immer zuerst den IDLE-Zustand.

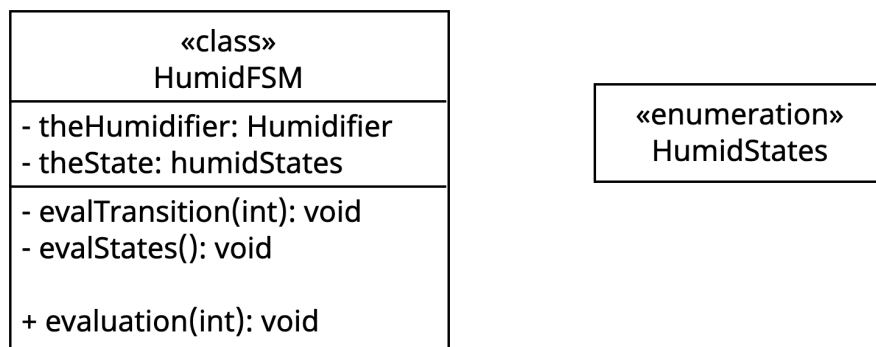


Abbildung 2: Klassendiagramm HumidFSM

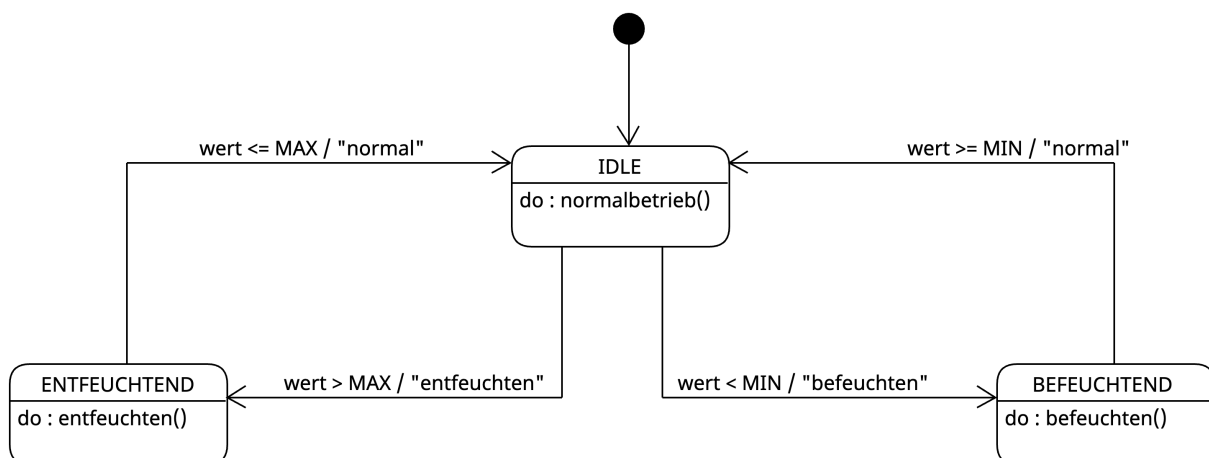


Abbildung 3: FSM-Diagramm

a) Implementieren Sie *HumidStates* als enum Klasse.

b) Implementieren Sie *HumidFSM*.

Attribute:

- Zustände (*theState*) der Zustandsmaschine
- Luftentfeuchter (*theHumidifier*) als Referenz auf einen Luftbefeuchter (*Humidifier*) der Steuerungslogik

Methoden:

- Methode *void evalTransition(int current_humidity)* wertet die Transitionen der FSM aus.
- Methode *void evalState()* führt die Aktionen der States der FSM aus.
- Methode *void evaluation(int current_humidity)* ruft die beiden anderen Evaluierungsmethoden in der richtigen Reihenfolge auf und gibt den aktuellen Zustand auf der Konsole aus.

c) Implementieren Sie Tests mit 100% Transitionsüberdeckung. Steigern Sie die Effizienz Ihrer Tests, indem Sie Grenzwerte als Testdaten einsetzen. Für die Implementierung der Tests nehmen Sie bitte die Testdatei ***testRaumueberwachungssystem.cpp***.

Aufgabe 3: Filehandling [40 Punkte]

Das Raumüberwachungssystem soll nun automatisiert getestet werden. Dafür wird die Klasse *FileHandler* benötigt, die die simulierten Daten für die Raumfeuchtigkeit aus einer Datei ausliest und anschließend der **HumidFSM** zu Auswertung übergibt.

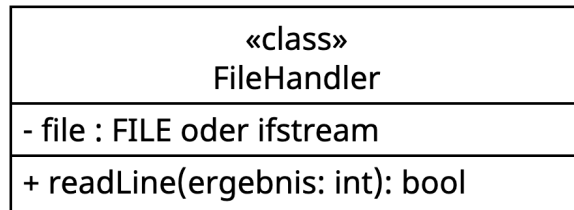


Abbildung 4: FileHandler-Klasse

Hinweis zur Implementierung: Es steht Ihnen frei, mit *Stream* oder *FILE* zu arbeiten. Passen Sie die Spezifikation der Klasse nach Ihrem Bedarf sinnvoll an.

- a) Als Attribut hat **FileHandler** die Datei, aus der die Daten ausgelesen werden sollen.
- b) Jede Zeile der Datei **humidityTestData.txt** beinhaltet einen Feuchtigkeitswert. Die Methode *readLine()* liest die Feuchtigkeitswerte zeilenweise aus. Der ausgelesene Wert wird in die per Referenz übergebene Variable *ergebnis* geschrieben. Der Rückgabewert der Methode gibt an, ob eine Zeile gelesen wurde (*true*) oder ob keine Zeile mehr gelesen werden konnte (*false*).
- c) In *main.cpp* sollen die ausgelesenen Feuchtigkeitswerte durch **HumidFSM** ausgewertet werden. Die Daten werden so lange ausgewertet, bis die Datei **humidityTestData.txt** am Ende ist.

Aufgabe 4: Interfaces [40 Punkte]

Das Raumüberwachungssystem bietet eine Schnittstelle *IDispatcher* an, die die Daten an unterschiedliche Überwachungseinheiten zur Verarbeitung weiterleitet, z.B. Temperatur, CO2-Gehalt, Raumfeuchtigkeit etc. In dem aktuellen System sollen Raumfeuchtigkeit und CO2-Gehalt eines Raumes gemessen werden.

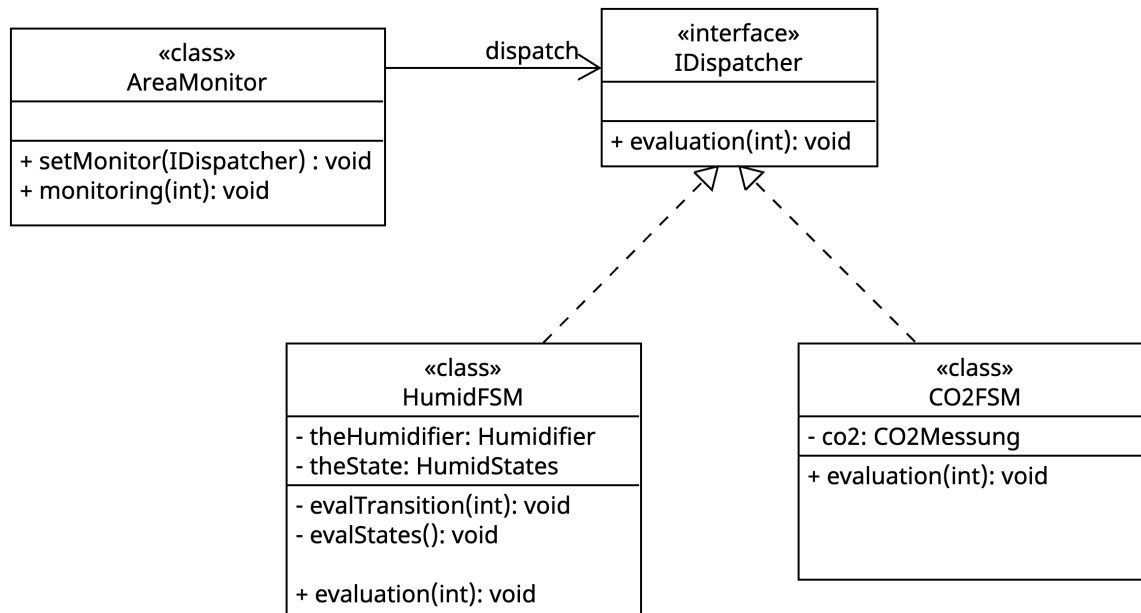


Abbildung 5: Klassendiagramm zum IDispatcher

- Implementieren Sie die Interface Klasse *IDispatcher*.
- Klasse **AreaMonitor**
 - Über die Methode *void setMonitor(IDispatcher)* wird das zu verwendende FSM-Objekt mitgeteilt.
 - Die Methode *void monitoring(int)* greift auf die Interface Methode *evaluation(int)* von **IDispatcher** zu.
- Wandeln Sie die Klassen **HumidFSM** und **CO2FSM** in Provider Klassen um.
- Implementieren Sie in der *main.cpp* das folgende Szenario: Führe zuerst das Monitoring von der *HumidFSM* mit den Inputs: 30 und 50 aus. Anschließend wird das Monitoring von *CO2FSM* mit den Inputs: 1000 und 2050 ausgeführt.

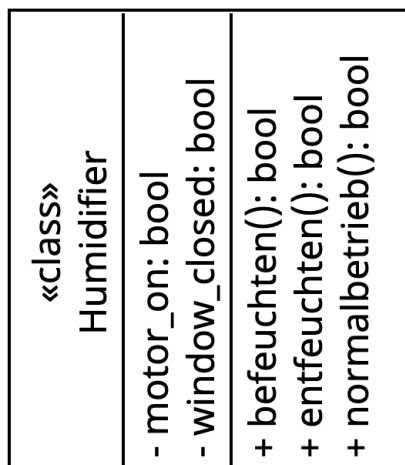


Abbildung 8: Klasse Humidifier

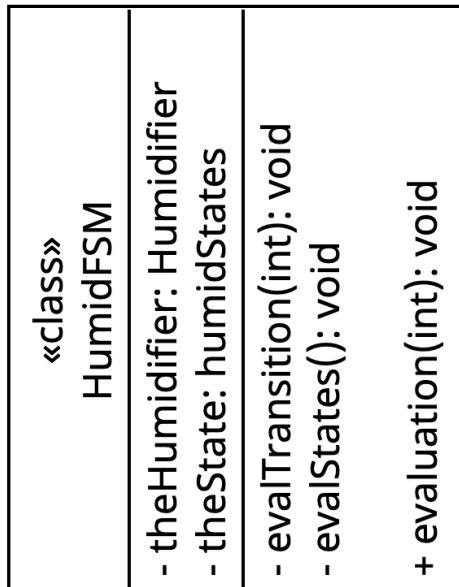


Abbildung 7: Klassendiagramm
HumidFSM

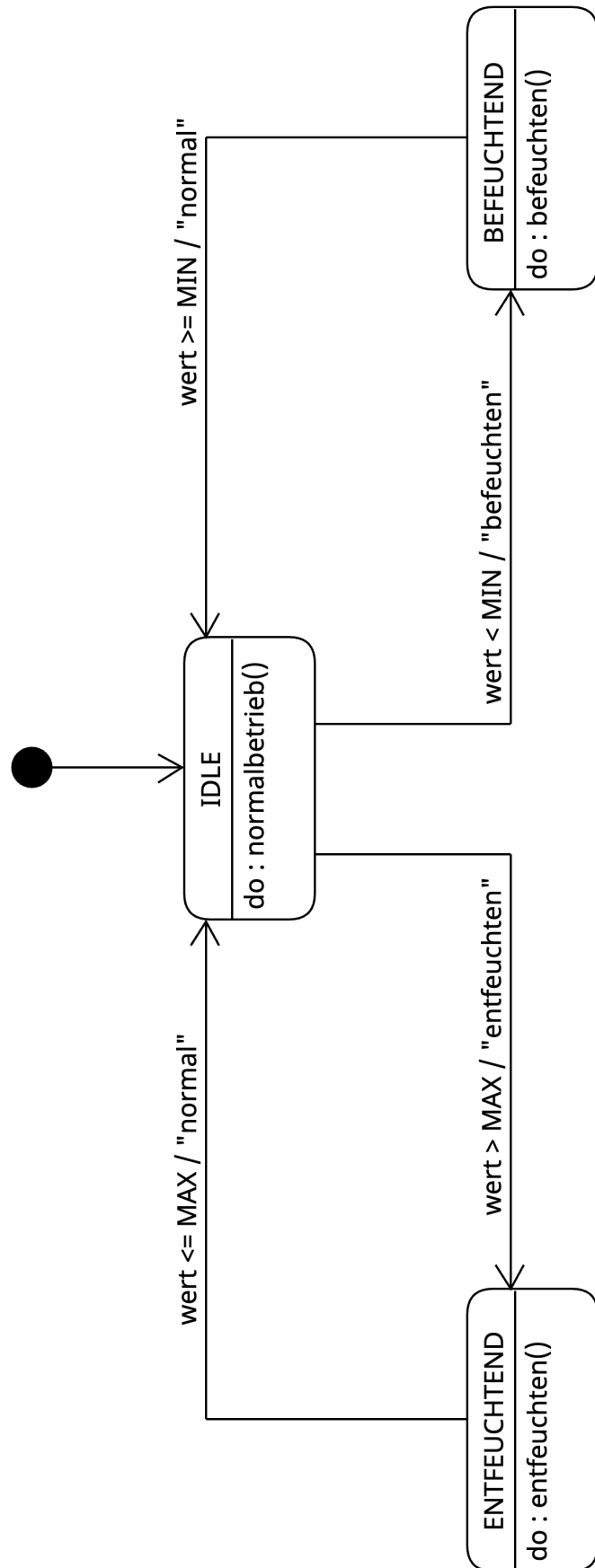
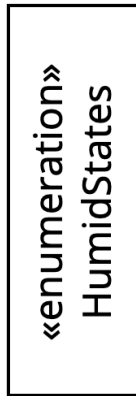


Abbildung 6: FSM-Diagramm

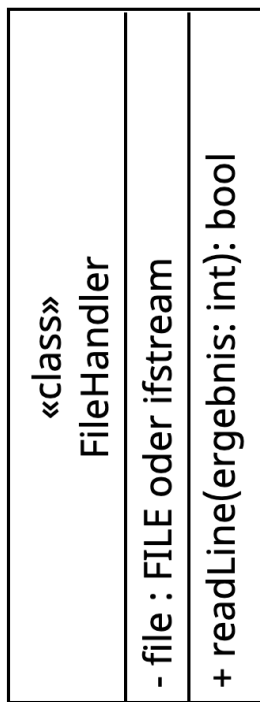
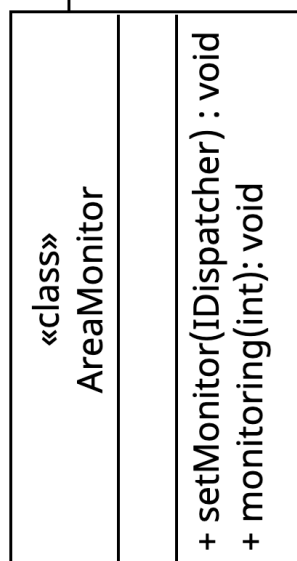


Abbildung 10: File-Handler-Klasse



dispatch

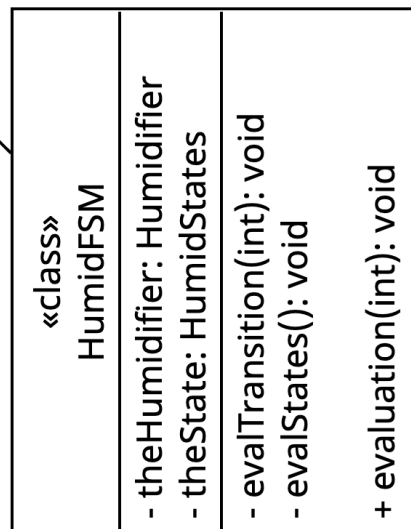
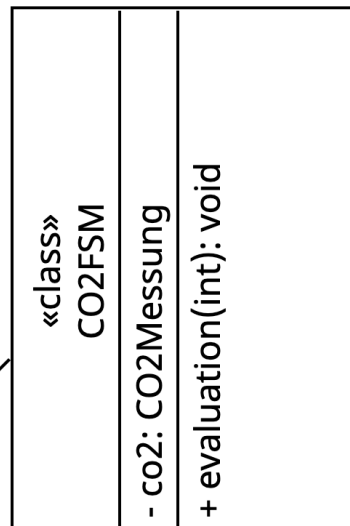


Abbildung 9: Klassendiagramm zum IDispatcher